

Guía de Calidad de Código

SonarQube para my-notes-js

Análisis estático de código JavaScript/CSS/HTML · Configuración paso a paso

Proyecto my-notes-js	Stack JS / CSS / HTML	Herramienta SonarQube	Versión guía Mayo 2026
--------------------------------	---------------------------------	---------------------------------	----------------------------------

1. ¿Qué es SonarQube y por qué usarlo?

SonarQube es una plataforma de análisis estático de código que detecta bugs, vulnerabilidades de seguridad, code smells y mide la cobertura de tests. Aunque my-notes-js es una aplicación frontend sencilla (vanilla JS + Bootstrap), SonarQube aporta valor real:

Categoría	Qué detecta en este proyecto
Bugs potenciales	Variables no declaradas, comparaciones incorrectas (== vs ===), referencias a null
Vulnerabilidades	Uso de innerHTML sin sanitización, exposición de datos en localStorage
Code Smells	Funciones muy largas, código duplicado, variables sin usar
Cobertura	Mide qué porcentaje del JS tiene tests (actualmente 0%, ideal $\geq 70\%$)
Deuda técnica	Calcula el tiempo estimado para refactorizar el código problemático

 **Importante SonarQube** tiene dos modalidades: SonarQube Community (self-hosted, gratis) y SonarCloud (cloud, gratis para proyectos públicos). Esta guía cubre ambas opciones.

2. Prerrequisitos

Antes de comenzar, verifica que tienes instalado:

Requisito	Para qué se usa
Node.js ≥ 18	Para crear package.json y correr scripts de análisis
Java 17+	Requerido por SonarQube Server (solo si usas modo self-hosted)
Docker (recomendado)	Forma más fácil de levantar SonarQube localmente
sonar-scanner CLI	Herramienta que envía el código al servidor. Se instala vía npm
Cuenta SonarCloud	Solo si usas la opción cloud (sonarcloud.io)

Verificar Java: `java -version` | **Verificar Node:** `node -v`

3. Clonar el repositorio

Si aún no tienes el proyecto en tu máquina local, sigue estos pasos:

3.1 Clonar desde GitHub

```
# Clonar el repositorio
git clone https://github.com/rueeen/my-notes-js.git

# Entrar a la carpeta del proyecto
cd my-notes-js

# Verificar que estás en la rama correcta
git branch
# Debería mostrar: * master
```

3.2 Verificar el contenido inicial

Después de clonar, el proyecto tiene esta estructura mínima (sin herramientas de calidad aún):

```
my-notes-js/
├── css/
│   └── styles.css      ← estilos del dashboard
├── js/
│   └── app.js          ← lógica principal (vanilla JS)
├── index.html          ← entrada de la app
└── (sin package.json) ← falta configuración de Node
```

i **Abrir la app localmente** Como es HTML puro, puedes abrir index.html directamente en el navegador, o usar Live Server en VS Code (extensión recomendada para desarrollo).

4. Estructura del proyecto con SonarQube

Después de seguir esta guía, el proyecto quedará con la siguiente estructura. Los archivos marcados con (+) son los que debes crear:

```
my-notes-js/
├── .github/
│   └── workflows/
│       └── sonar.yml          (+) pipeline CI/CD automático
├── css/
│   └── styles.css
├── js/
│   ├── app.js
│   └── app.test.js           (+) tests unitarios con Jest
├── coverage/                 (+) tests unitarios con Jest
│   └── lcov.info             (+) (generada) reporte de cobertura
├── node_modules/             (+) (generada) consumida por SonarQube
├── index.html                (+) (generada) dependencias instaladas
├── jest.config.js            (+) configuración de Jest
├── package.json              (+) scripts y dependencias
├── package-lock.json         (+) (generada) al correr npm install
└── sonar-project.properties  (+) configuración de SonarQube
```

🚫 **Agregar al .gitignore** Crea un archivo .gitignore si no existe y agrega estas líneas:
node_modules/ coverage/ .sonar/
Nunca commitear node_modules ni archivos de cobertura generados.

5. Flujo completo de comandos

Una vez clonado el repo y creados los archivos de configuración, estos son todos los comandos en orden:

5.1 Primera vez (setup inicial)

```
# 1. Entrar al proyecto
cd my-notes-js

# 2. Inicializar npm (o crear package.json manualmente según sección 6)
npm init -y

# 3. Instalar dependencias de desarrollo
npm install --save-dev sonar-scanner jest jest-environment-jsdom

# 4. Verificar instalación
npx sonar-scanner --version
npx jest --version
```

5.2 Correr tests y generar cobertura

```
# Correr tests una vez
npm test

# Correr tests en modo watch (desarrollo)
npm test -- --watch

# Correr solo cobertura sin watch
npm test -- --coverage --watchAll=false

# Verificar que se generó el reporte
ls coverage/
# Debe mostrar: lcov.info lcov-report/ ...
```

5.3 Ejecutar el análisis de SonarQube

```
# Definir el token como variable de entorno (Mac/Linux)
export SONAR_TOKEN=sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

# Definir el token (Windows PowerShell)
$env: SONAR_TOKEN="sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"

# Correr el análisis completo (tests + sonar)
npm test -- --coverage --watchAll=false && npm run sonar

# O por separado:
npm test
npm run sonar
```

5.4 Flujo de trabajo diario (después del setup)

```
# 1. Hacer cambios en el código

# 2. Correr tests para verificar que todo sigue verde
npm test

# 3. Correr el análisis de SonarQube
npm run sonar

# 4. Revisar resultados en el dashboard:
#   SonarCloud: https://sonarcloud.io/project/overview?id=my-notes-js
#   Local:      http://localhost:9000/dashboard?id=my-notes-js

# 5. Corregir los issues detectados y repetir desde el paso 2
```

5.5 Comandos útiles de diagnóstico

```
# Ver qué archivos detecta el scanner antes de analizarlos
npx sonar-scanner -Dsonar.scanner.dumpToFile=sonar-dump.txt

# Limpiar caché del scanner si hay problemas
rm -rf .scannerwork/

# Ver logs detallados del análisis
npx sonar-scanner -X

# Verificar conectividad con SonarCloud
curl -u $SONAR_TOKEN: https://sonarcloud.io/api/system/status
```

6. Archivos que debes agregar al proyecto

El repositorio actual no tiene ningún archivo de configuración para herramientas de calidad. Estos son los 3 archivos necesarios:

6.1 package.json

Actualmente el proyecto no tiene package.json. Es necesario para instalar el scanner y correr scripts de análisis:

```
{
  "name": "my-notes-js",
  "version": "1.0.0",
  "description": "Post-it Notes Dashboard",
  "scripts": {
    "sonar": "sonar-scanner",
    "test": "jest --coverage",
    "test:ci": "jest --coverage --ci"
  },
  "devDependencies": {
    "sonar-scanner": "^3.1.0",
    "jest": "^29.0.0",
    "jest-environment-jsdom": "^29.0.0"
  }
}
```

6.2 sonar-project.properties

Archivo principal de configuración. SonarQube lo lee para saber qué analizar:

```
# — Identificación del proyecto —————
sonar.projectKey=my-notes-js
sonar.projectName=My Notes JS
sonar.projectVersion=1.0

# — Código a analizar —————
sonar.sources=js,css
sonar.inclusions=js/**/*.js, css/**/*.css,index.html
sonar.exclusions=**/node_modules/**/*.js,**/coverage/**

# — Cobertura de tests (cuando los tengas) —————
sonar.javascript.lcov.reportPaths=coverage/lcov.info

# — Encoding —————
sonar.sourceEncoding=UTF-8
```

6.3 jest.config.js (para cobertura de tests)

Configura Jest para que genere el reporte de cobertura que SonarQube consume:

```
module.exports = {
  testEnvironment: "jsdom",
  collectCoverageFrom: ["js/**/*.js"],
  coverageReporters: ["lcov", "text-summary"],
  coverageDirectory: "coverage"
};
```

7. Opciones de instalación

A	SonarCloud (recomendado)	Gratis para repositorios públicos. Sin infraestructura. Ideal para este proyecto.
B	SonarQube + Docker	Self-hosted local. Más control, requiere Docker corriendo.
C	SonarQube descarga directa	Sin Docker. Requiere Java 17+ instalado manualmente.

Opción A — SonarCloud (más fácil para proyectos públicos)

1. **Crear cuenta en SonarCloud** → Ve a sonarcloud.io y entra con tu cuenta de GitHub.
2. **Importar repositorio** → Click en "+" > Analyze new project > selecciona rueeen/my-notes-js.
3. **Generar token** → My Account > Security > Generate Token. Guárdalo como SONAR_TOKEN.
4. **Agregar al sonar-project.properties:**

```
sonar.host.url=https://sonarcloud.io
sonar.organization=rueeen
sonar.token=TU_TOKEN_AQUI # ← nunca commitear esto directamente
```

⚠ Para no exponer el token en el repositorio, defínelo como variable de entorno:

```
# En tu terminal (Mac/Linux):
export SONAR_TOKEN=sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX

# En Windows (PowerShell):
$env:SONAR_TOKEN="sqp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
```

5. **Instalar y correr el scanner:**

```
npm install
npm run sonar
```

El análisis tardará ~1-2 minutos. Al terminar, SonarCloud te dará un link con el reporte.

Opción B — SonarQube con Docker (self-hosted)

6. Levantar SonarQube con Docker Compose:

```
# Crea docker-compose.yml en cualquier carpeta:
version: '3'
services:
  sonarqube:
    image: sonarqube:community
    ports:
      - "9000:9000"
    environment:
      - SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true
    volumes:
      - sonarqube_data:/opt/sonarqube/data
volumes:
  sonarqube_data:
docker compose up -d
# Esperar ~60 segundos, luego abrir http://localhost:9000
# Login: admin / admin (cambia la contraseña al entrar)
```

7. **Crear proyecto manualmente** → En la UI web: Create Project > Manually > nombre: my-notes-js.

8. **Generar token** → En la configuración del proyecto > Generate token.

9. **Agregar al sonar-project.properties:**

```
sonar.host.url=http://localhost:9000
sonar.token=TU_TOKEN_LOCAL
```

10. **Correr el scanner:**

```
npm install
npm run sonar
```

8. Agregar tests para cobertura real

SonarQube puede analizar el código sin tests, pero la métrica más importante —cobertura— quedará en 0%. Aquí un ejemplo de tests básicos para el módulo principal de app.js:

Crea el archivo js/app.test.js:

```
/**
 * Tests básicos para my-notes-js
 * Ejecutar: npm test
 */

// Mock de localStorage
const localStorageMock = (() => {
  let store = {};
  return {
    getItem: (k) => store[k] || null,
    setItem: (k, v) => { store[k] = String(v); },
    removeItem: (k) => { delete store[k]; },
    clear: () => { store = {}; }
  };
})();
Object.defineProperty(window, 'localStorage', { value: localStorageMock });

describe('Almacenamiento de notas', () => {
  beforeEach(() => localStorage.clear());

  test('guarda una nota correctamente', () => {
    const nota = { id: 1, title: 'Test', description: 'Desc', important: false };
    localStorage.setItem('notes', JSON.stringify([nota]));
    const stored = JSON.parse(localStorage.getItem('notes'));
    expect(stored).toHaveLength(1);
    expect(stored[0].title).toBe('Test');
  });

  test('notas importantes tienen flag correcto', () => {
    const nota = { id: 2, title: 'Urgente', important: true };
    expect(nota.important).toBe(true);
  });
});
```

Correr tests con cobertura:

```
npm test
# Genera coverage/lcov.info → SonarQube lo leerá automáticamente
```

9. Errores intencionales en el código

Para que los estudiantes vean el valor real de SonarQube, el repositorio incluye errores plantados intencionalmente en `js/app.js`. Cada error pertenece a una categoría distinta del análisis estático:

Categoría SonarQube	Error plantado
BUG	Comparación débil con <code>==</code> en lugar de <code>===</code> al verificar el tipo de nota
BUG	Acceso a propiedad de objeto potencialmente null sin verificación previa
VULNERABILIDAD	innerHTML sin sanitización — permite inyección XSS con notas maliciosas
CODE SMELL	Variable global declarada con <code>var</code> fuera de cualquier función o módulo
CODE SMELL	<code>console.log</code> de depuración dejado en el código de producción
CODE SMELL	Función duplicada — lógica de renderizado copiada con cambio mínimo
CODE SMELL	Código muerto — bloque después de <code>return</code> que nunca se ejecuta
CODE SMELL	Función excesivamente larga con más de 30 líneas y múltiples responsabilidades

📌 **Objetivo pedagógico** Los estudiantes deben: (1) correr SonarQube, (2) identificar cada issue en el reporte, (3) ubicar el código problemático, y (4) proponer y aplicar la corrección. El archivo `js/app.js` tiene comentarios `// SONAR-ISSUE:` que marcan cada error.

Mapa de errores en el archivo js/app.js

Ubicación	Descripción del issue
Línea ~15	var notesCache = [] → variable global con var (CODE SMELL)
Línea ~28	console.log('DEBUG...') → log de depuración (CODE SMELL)
Línea ~45	if (note.important == true) → comparación débil (BUG)
Línea ~52	element.innerHTML = userInput → XSS sin sanitizar (VULNERABILIDAD)
Línea ~67	return result; /* código muerto después */ → código muerto (BUG)
Línea ~80	function renderNoteCard() {...} → duplicado de renderNote() (CODE SMELL)
Línea ~110	nota.author.toUpperCase() → posible null reference (BUG)
Línea ~120	function processAndRenderAllNotes() → función de 35+ líneas (CODE SMELL)

10. Interpretar el reporte de SonarQube

Una vez ejecutado el análisis, el dashboard muestra estas métricas clave:

Métrica	Qué significa
Quality Gate	Estado general: PASSED ✓ o FAILED X. Es el semáforo principal del proyecto.
Bugs	Errores de lógica que podrían causar comportamiento incorrecto en runtime.
Vulnerabilities	Problemas de seguridad explotables. En JS frontend: XSS, eval(), innerHTML sin escape.
Security Hotspots	Código que requiere revisión manual. No es bug confirmado, pero merece atención.
Code Smells	Problemas de mantenibilidad: funciones muy largas, código duplicado, etc.
Coverage	% de líneas de código cubiertas por tests. Meta recomendada: $\geq 70\%$.
Duplications	% de código duplicado. Meta: $< 3\%$.
Technical Debt	Tiempo estimado para resolver todos los code smells (ej: "2h 30min").

Quality Gate por defecto (Sonar Way)

Para pasar el Quality Gate en código nuevo, SonarQube exige:

- Cobertura en código nuevo $\geq 80\%$
- 0 Bugs nuevos
- 0 Vulnerabilidades nuevas
- Duplicación en código nuevo $< 3\%$
- Security Hotspots revisados: 100%

11. Problemas comunes en proyectos JS vanilla

Problema detectado	Cómo corregirlo
innerHTML sin escape	SonarQube lo marca como vulnerabilidad XSS. Usar <code>textContent</code> o sanitizar con <code>DOMPurify</code> .
Variables globales (var)	Declarar con <code>let</code> o <code>const</code> dentro de funciones/módulos.
== en lugar de ===	Comparación débil. Reemplazar por <code>===</code> en todo el código.
Funciones anónimas largas	Extraer en funciones nombradas. Mejora legibilidad y trazabilidad.
console.log en producción	SonarQube lo marca como code smell. Remover o usar un logger condicional.
Código no alcanzable	Código después de <code>return/throw</code> . SonarQube lo detecta y marca como bug.

12. Resumen: checklist completo

✅ **Archivos a crear en el repo** package.json — scripts de sonar y test
sonar-project.properties — configuración del proyecto
jest.config.js — cobertura de tests
js/app.test.js — al menos tests básicos
.gitignore — excluir node_modules/ y coverage/

🔑 **Variables de entorno necesarias** SONAR_TOKEN — token generado en SonarCloud o SonarQube local
Definir en la terminal antes de correr npm run sonar
Mac/Linux: export SONAR_TOKEN=sqp_xxx | Windows PS:
\$env:SONAR_TOKEN='sqp_xxx'

🎯 **Meta de calidad inicial recomendada** Coverage $\geq 50\%$ (subir progresivamente a 70%+)
0 Bugs y 0 Vulnerabilidades
Deuda técnica < 1 día
Duplicación < 5%

github.com/rueeen/my-notes-js · Guía generada Mayo 2026